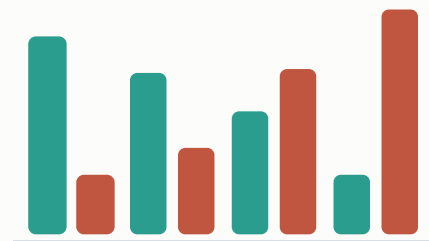


Text Classification with Multinomial Naive Bayes

 Week 7 Laboratory · 2 hours



1

Learn priors and
likelihoods

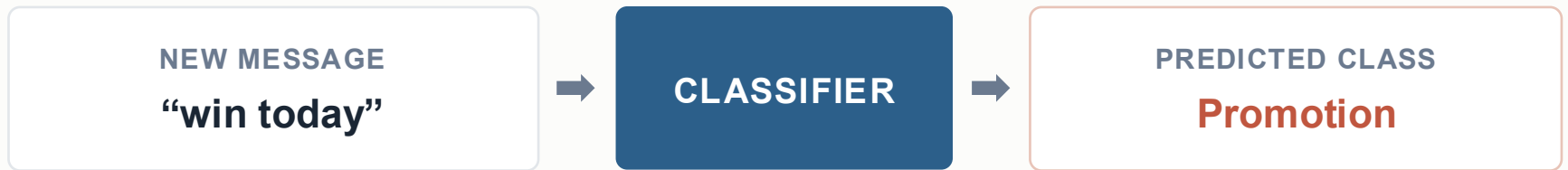
2

Predict a message
by hand

3

Smoothing and log
scores

A Text Classifier



INPUT

POSSIBLE CLASSES

SMS



Normal / Promotion

Email



Spam / Ham (not spam)

Review



Positive / Negative

Text as Training Data

A training example = (Text input, Label)

MESSAGE

1. team meeting today
2. project meeting today
3. team project update
4. win cash today
5. win cash cash prize

Text!

LABEL

Normal

Normal

Normal

Promotion

Promotion

QUESTION

Which words look like evidence for **Promotion**?

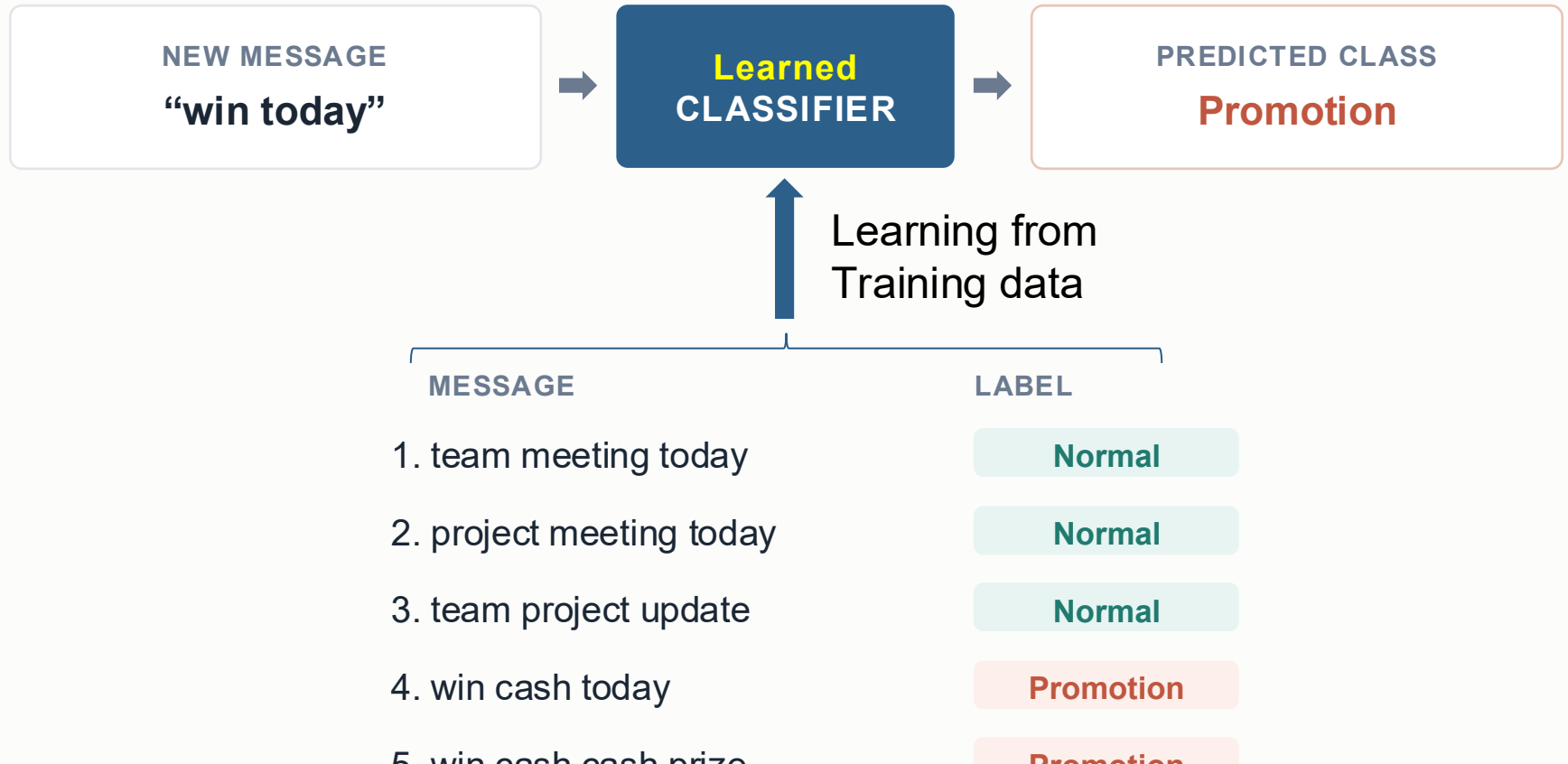
win

cash

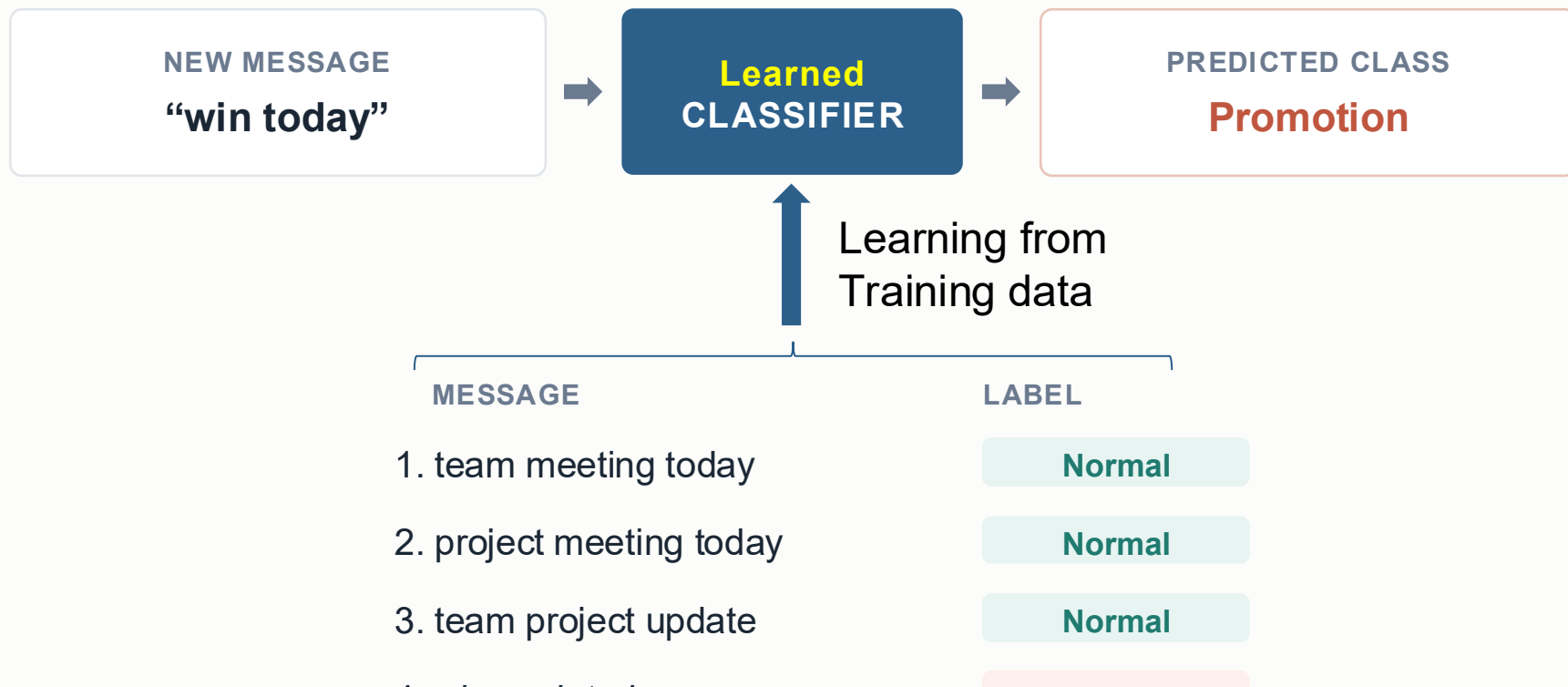
prize

These three words never appear in a Normal message.

A Text Classifier learned from **Training Data**



Multinomial Naïve Bayes **as a Classifier**



Multinomial Naïve Bayes

Bayes: Update a Prior Belief Using Evidence

$$\begin{array}{c} \text{POSTERIOR} \\ \hline P(\text{class} \mid d) \end{array} = \frac{\begin{array}{c} \text{LIKELIHOOD} \\ \hline P(d \mid \text{class}) \end{array} \times \begin{array}{c} \text{PRIOR} \\ \hline P(\text{class}) \end{array}}{\begin{array}{c} P(d) \\ \text{Evidence/Marginal Prob} \end{array}}$$

P(class) Prior: our prior belief of a class before observing any document

P(class | d) **Posterior:** Given a document **d**, what is the probability it belongs to a **class**?

Simply, $P(c \mid d) \propto P(c) \times P(d \mid c)$ because $P(d)$ is identical for every class

Multinomial Naïve Bayes

Naïve: Treat Words as Conditionally Independent

Message d: “win cash cash prize”

win

cash

cash

prize

*cash appears
twice*

$$P(\text{message} \mid c) \approx \prod P(w_i \mid c)$$

$$P(\text{win} \mid c) \times P(\text{cash} \mid c) \times P(\text{cash} \mid c) \times P(\text{prize} \mid c)$$

Real words are related — “cash prize” is hardly independent — but this simplifying assumption is fast and works surprisingly well.

Multinomial Naïve Bayes

Text Must Be Represented Numerically

Every message becomes a vector of word counts over a fixed vocabulary.

VOCABULARY $|V| = 8$

team	meeting	today	project	update	win	cash	prize
------	---------	-------	---------	--------	-----	------	-------

COUNTS FOR THE MESSAGE “win cash cash today”

0	0	1	0	0	1	2	0
---	---	---	---	---	---	---	---

Bag-of-Words vector: **[0, 0, 1, 0, 0, 1, 2, 0]**

Multinomial Naïve Bayes

Multinomial: Use How Many Times Each Word Occurs

“win cash cash today”

team	meeting	today	project	update	win	cash	prize
0	0	1	0	0	1	2	0

The feature values are counts, not just *yes / no* (Bernoulli – One-hot).

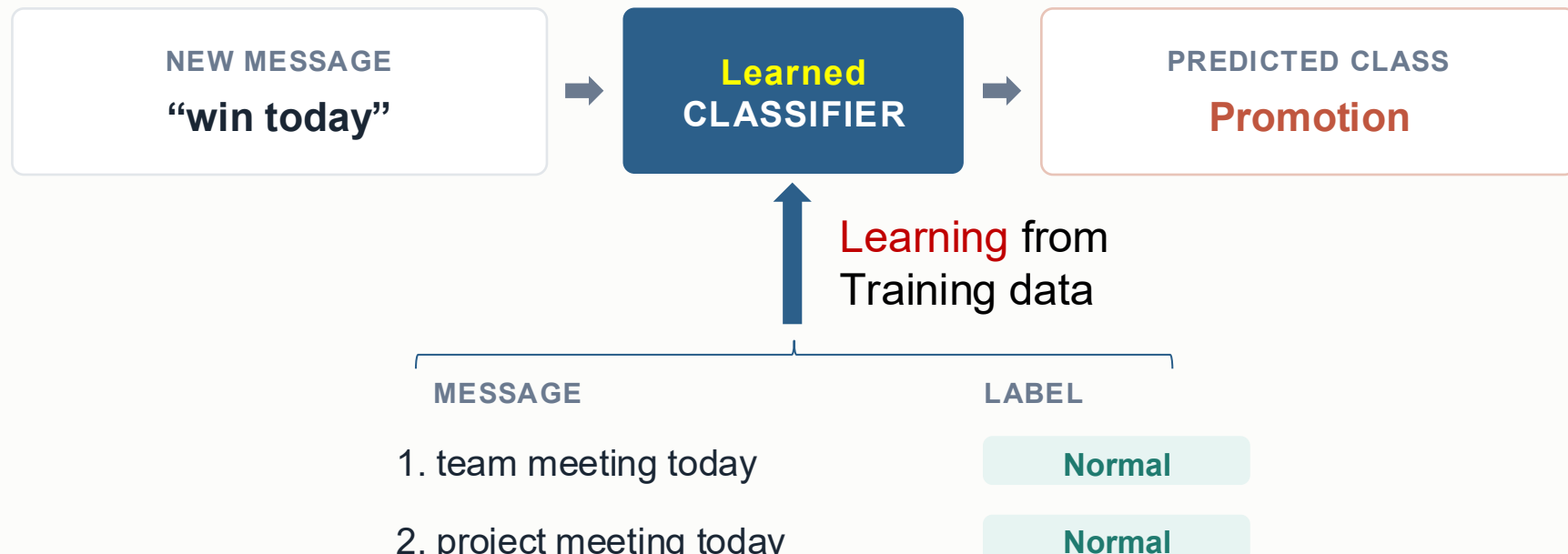
Bag-of-Words vector: [0, 0, 1, 0, 0, 1, 2, 0]

TAKEAWAY Multinomial NB models word counts — it does not mean the task must have many classes.

Training a Multinomial Naïve Bayes classifier

$$\overbrace{P(\text{class} | d)}^{\text{POSTERIOR}} = \frac{\overbrace{P(d | \text{class})}^{\text{LIKELIHOOD}} \times \overbrace{P(\text{class})}^{\text{PRIOR}}}{P(d)}$$

Learning what? $\left\{ \begin{array}{l} P(\text{class } c_i) \\ P(d | \text{class } c_i) \end{array} \right.$



Training: Step 1 — Learn How Common Each Class Is?

The *class prior* $P(c)$ is estimated by counting documents.

Normal

3 of 5 documents



0.60

$P(\text{normal}) = 3 / 5$

Promotion

2 of 5 documents



0.40

$P(\text{promotion}) = 2 / 5$

1. team meeting today

Normal

2. project meeting today

Normal

3. team project update

Normal

4. win cash today

Promotion

5. win cash cash prize

Promotion

$$P(c) = \text{number of documents in class } c \div \text{total number of documents}$$

Training: Step 2 — Count Word Occurrences in Each Class

Every token in every message is counted, class by class.

WORD	NORMAL	PROMO.
team	2	0
meeting	2	0
today	2	1
project	2	0

WORD	NORMAL	PROMO.
update	1	0
win	0	2
cash	0	3
prize	0	1

1. team meeting today

Normal

2. project meeting today

Normal

3. team project update

Normal

4. win cash today

Promotion

5. win cash cash prize

Promotion

Training: Step 3 — Estimate Word Likelihoods

$P(w | c)$ answers: how likely is word w inside documents of class c ?

$$P(w | c) = \text{count of word } w \text{ in class } c \div \text{total tokens in class } c$$

WITHOUT SMOOTHING

$P(\text{team} | \text{Normal})$

$$2 / 9 \approx 0.22$$

$P(\text{win} | \text{Promotion})$

$$2 / 7 \approx 0.29$$

$P(\text{win} | \text{Normal})$

$$0 / 9 = 0$$

PROBLEM A single zero likelihood drives the whole class score to zero: $P(\text{Normal}) \times 0 \times \dots = 0$?
e.g., a message “see you tomorrow”

Laplace Smoothing Prevents Zero Probabilities

Add one to every count, and add $|V|$ to every denominator.

$$P(w | c) = \frac{\text{count}(w, c) + \alpha}{\text{total tokens}(c) + |V|}$$

NEW DENOMINATORS

$(\alpha = 1, |V| = 8)$

Normal

9 + 8 = 17

Promotion

7 + 8 = 15

$P(\text{win} | \text{Promotion}) = (2+1) / (7+8) = 3/15$

WORD

NORMAL

PROMOTION

win

1 / 17

3 / 15

cash

1 / 17

4 / 15

today

3 / 17

2 / 15

meeting

3 / 17

1 / 15

Thanh

1 / 8

1 / 8

Unseen words receive a small probability, **instead of 0**.

Prediction: Update Prior with Observations

$$\begin{array}{ccc} \text{POSTERIOR} & \text{LIKELIHOOD} & \text{PRIOR} \\ \hline P(\text{class} | d) & = & \frac{P(d | \text{class}) \times P(\text{class})}{P(d)} \end{array}$$

New message: “win cash cash today”

$$\text{score}(c) = P(c) \times P(\text{win} | c) \times P(\text{cash} | c)^2 \times P(\text{today} | c)$$

score(Normal)

$$0.60 \times 1/17 \times (1/17)^2 \times 3/17$$

≈ 0.0000216

score(Promotion)

LARGER SCORE

$$0.40 \times 3/15 \times (4/15)^2 \times 2/15$$

≈ 0.000759

Normal has the larger prior, but “win” and “cash” are far stronger evidence for Promotion.

Why Use Logarithms?

Multiplying many probabilities creates extremely small numbers.

PROBLEM For a long document, $0.02 \times 0.01 \times 0.03 \times \dots$ becomes too small for a computer to represent accurately.

THE LOGARITHM IDENTITY

$$\log(a \times b) = \log(a) + \log(b)$$

$$\log \text{ score}(c) = \log P(c) + \sum \log P(w | c)$$

No numerical underflow

Multiplication becomes
addition

The predicted class is
unchanged

WHY IT WORKS The logarithm is monotonic, so the largest score is still the largest log score.

The Same Prediction in Log Space

Same message, same winner — now computed by adding instead of multiplying.

score(Normal) ≈ 0.0000216

log score(Normal)

$\log(0.60) + \log(1/17)$
 $+ 2 \cdot \log(1/17) + \log(3/17)$

≈ -10.75

score(Promotion) ≈ 0.000759

log score(Promotion)

$\log(0.40) + \log(3/15)$
 $+ 2 \cdot \log(4/15) + \log(2/15)$

≈ -7.18

$-7.18 > -10.75$ so the prediction is still Promotion

Again, Where the Naive Assumption Breaks Down?

The assumption is wrong in general — and still useful in practice.

$$P(\text{class} \mid d) = \frac{P(d \mid \text{class}) \times P(\text{class})}{P(d)}$$

THE ASSUMPTION

$P(\text{win, cash, cash, today} \mid \text{Promotion})$

is approximated by

$P(\text{win} \mid \text{Promotion}) \times P(\text{cash} \mid \text{Promotion})^2 \times P(\text{today} \mid \text{Promotion})$

BUT REAL WORDS ARE NOT INDEPENDENT

“credit card”

“machine learning”

“not good”

The assumption is unrealistic, yet Naive Bayes is still a **strong text baseline**.

Training v.s. Prediction

TRAINING

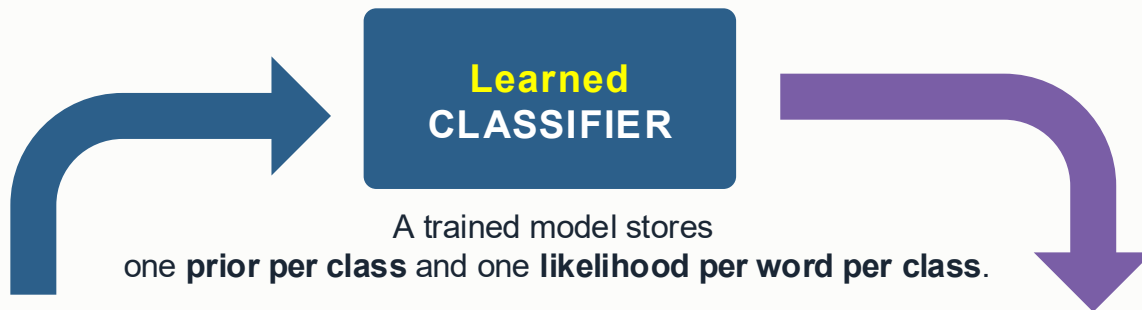
Learns from labelled data
Estimates the probabilities
Happens when the model is fitted



PREDICTION

- Applies the learned model
- Calculates a score per class
- Happens when a message is predicted

Training Learns the Probabilities; Prediction Uses Them



Training

INPUT

Labelled training documents

LEARN

$P(c)$ per class, $P(w | c)$ per word

STORE

Log class priors and log word likelihoods

Prediction

INPUT

One new, unlabelled document

CALCULATE

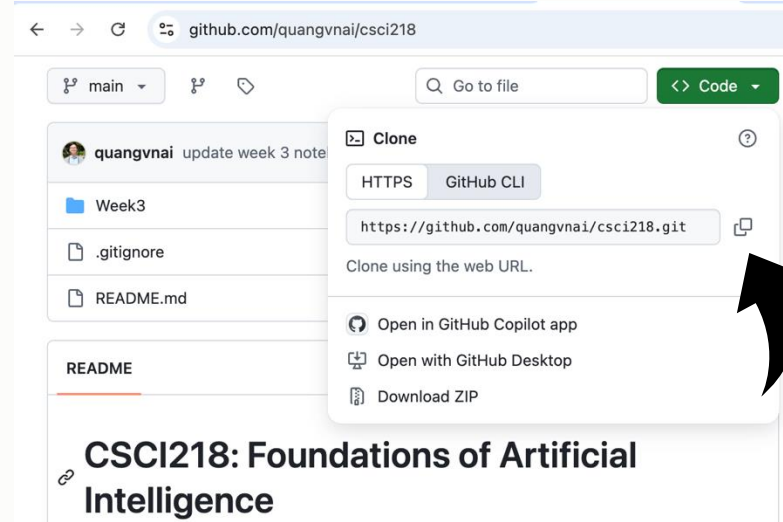
$\log P(c) + \sum x_v \cdot \log P(w | c)$ for each class

OUTPUT

The class with the largest log score

Our CSCI218 Github Repository

```
git clone https://github.com/quangvnai/csci218  
  
cd csci218
```



Lab Task: Naive Bayes for Text Classification

→ Download the Week 7 notebook and open it in “csci218/week7”

OPEN THE WEEK 7 NOTEBOOK → train and evaluate Multinomial Naive Bayes

- ✓ Build the vocabulary and turn each message into a Bag-of-Words count vector.
- ✓ Estimate class priors and Laplace-smoothed word likelihoods from the training set.
- ✓ Predict a class by summing log probabilities.
- ✓ Compare your from-scratch model with scikit-learn's MultinomialNB.
- ✓ Evaluate with accuracy, precision, recall, F1-score and a confusion matrix.

Dataset: 20 Newsgroups

A classic benchmark for text classification



~18,000 ENGLISH POSTS → 20 DISCUSSION GROUPS

- ✓ 20 balanced classes spanning science, computing, politics, religion and recreation.
- ✓ Raw text with headers, quotes and signatures — light cleaning matters before Bag-of-Words.
- ✓ Ships with scikit-learn: `fetch_20newsgroups()` gives ready train / test splits.

WRAP-UP



Review & Reflection

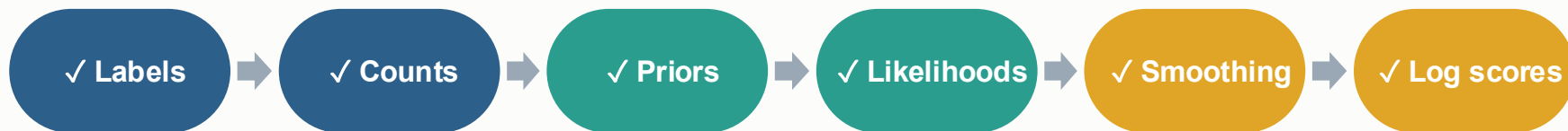


IN THIS SECTION

What we learned · Reflection questions

What Have We Learned?

One toy dataset, the whole Naive Bayes pipeline — every stage now checked off.



You built an entire Naive Bayes classifier by hand — end to end.

...and you know why smoothing and log probabilities are needed.

Reflection Questions

Before you leave, make sure you can answer these:

- 1 What exactly does a Naive Bayes model learn during training?
- 2 Why does one zero likelihood ruin an entire class score?
- 3 What does Laplace smoothing change, and why $|V|$ in the denominator?
- 4 Why do we add log probabilities instead of multiplying probabilities?
- 5 Which assumption makes Naive Bayes “naive”, and why does it still work?